

“Collaborative Story/Screenplay Writing Platform with Version History”

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Artificial Intelligence & Data Science

Submitted by

192424179- LITHISH KUMAR. R

Under the Supervision of

Dr. K Anita Davamani



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

August 2025



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, **Lithish Kumar R 192424179**, of the **Department of Artificial Intelligence and Data Science**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **“Collaborative Story/Screenplay Writing Platform with Version History”** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date:

Signature of the Students

ABSTRACT

Screenplay and long-form narrative writing is increasingly a collaborative endeavour, involving writers, editors, and reviewers who must coordinate revisions, maintain formatting consistency, and track the evolution of a script over time. Conventional word processors and generic document editors, however, are not designed for the specific structural and formatting conventions of screenplays, and they offer limited support for concurrent editing, version tracking, or role-based collaboration. This gap often results in fragmented workflows, formatting errors, and difficulty reconciling contributions from multiple collaborators. This case study presents the design and implementation of a web-based Collaborative Story/Screenplay Writing Platform intended to address these limitations within an academic capstone context.

The proposed system enables multiple users to edit a screenplay concurrently while automatically applying industry-standard screenplay formatting. It incorporates version history and alternate draft management, allowing writers to preserve and compare earlier revisions without requiring complex branch-merging mechanisms. Additional functionality includes inline commenting, character profile management, scene metadata organisation, project analytics, role-based access control, and search and filtering capabilities. The platform further integrates AI-assisted features such as grammar checking, formatting assistance, dialogue suggestions, character consistency checks, and automated scene summarisation, alongside export functionality for PDF and FDX formats. Real-time synchronisation is achieved through established collaborative editing technologies rather than a custom-built concurrency engine, reflecting the project's scope as an academic prototype rather than a production system.

The report documents the requirements analysis, system architecture, design decisions, and evaluation methodology underlying the platform's development. Expected outcomes include a functional prototype demonstrating reliable multi-user collaboration, formatting accuracy, and usable version control, alongside an assessment of the system's strengths and limitations. The study aims to contribute practical insight into the design of domain-specific collaborative writing tools within a software engineering education setting.

TABLE OF CONTENTS

S. No	TOPIC	PAGE NO
1	ABSTRACT	I
2	INTRODUCTION	1-6
3	PROBLEM IDENTIFICATION AND ANALYSIS	7-10
4	SYSTEM REQUIREMENTS SPECIFICATION (SRS)	10-12
5	SOLUTION DESIGN AND SYSTEM ANALYSIS	13-15
6	RESULTS AND RECOMMENDATION	16-17
7	REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT	18-19
8	CONCLUSION	20-22
9	REFERENCES	23

CHAPTER 1: INTRODUCTION

1.1 Introduction

Screenplay writing has traditionally been an individual or tightly controlled group activity, structured around discrete drafts that pass sequentially between a small number of contributors. As film, television, and digital media production have grown in scale and complexity, the number of stakeholders involved in shaping a screenplay — writers, script editors, story consultants, and producers — has increased correspondingly. Despite this shift toward collaborative authorship, the tools most commonly used for screenplay development have not kept pace with the way teams actually work. Word processors and dedicated screenplay formatting applications remain oriented toward single-user editing, with collaboration handled as an afterthought through file sharing, email attachments, or manual merging of changes.

This chapter introduces the capstone project, a browser-based collaborative screenplay writing platform designed to support multiple contributors working on the same screenplay concurrently, while preserving the structural conventions specific to screenplay formatting and the editorial workflows specific to script development. The chapter establishes the background and context that motivate the project, identifies the problem being addressed, and defines the aim, objectives, and scope that guide the remainder of the report.

1.2 Background

Collaborative software authorship has a long history in domains such as source code management, where version control systems evolved from simple file locking mechanisms to distributed systems capable of merging concurrent changes from many contributors. Collaborative document editing followed a comparable trajectory: early word processors supported no concurrent access at all, later systems introduced file locking or "track changes" style markup, and modern web-based editors (such as Google Docs) introduced real-time concurrent editing using operational transformation and, more recently, Conflict-free Replicated Data Types (CRDTs).

Screenplay writing sits at the intersection of creative authorship and structured document formatting. Unlike general prose, a screenplay follows strict industry-standard formatting conventions — scene headings, action lines, character cues, dialogue, and parentheticals — each governed by

specific indentation, capitalization, and pagination rules. These conventions exist because a correctly formatted screenplay page corresponds, by industry convention, to approximately one minute of screen time, which producers and production planners rely on for scheduling and budgeting purposes [citation needed]. This formatting requirement distinguishes screenplay editors from general-purpose word processors and has historically justified the existence of specialized tools such as Final Draft and Celtx.

The rise of remote and distributed creative teams, accelerated by broader trends in remote work over the past decade, has placed additional pressure on screenplay development workflows. Writers, editors, and producers are frequently distributed across different locations and time zones, and screenplay development often proceeds through multiple parallel drafts, alternate scene versions, and iterative feedback cycles that are difficult to coordinate using single-user desktop applications and manual file exchange.

1.3 Industry Context

The broader software industry has already demonstrated that real-time collaborative editing is technically achievable and commercially viable, primarily through general-purpose document platforms. Frameworks and libraries such as Yjs, Automerge, and hosted services such as Liveblocks have made CRDT-based synchronization accessible to application developers without requiring each team to design a custom conflict-resolution algorithm. These tools handle the underlying complexity of merging concurrent edits, allowing application developers to focus on domain-specific features built on top of the synchronization layer.

At the same time, the screenplay and script-writing segment of the software industry has remained comparatively conservative in adopting these advances. Established screenplay software still centers on a single-user editing model, with collaboration added through supplementary cloud storage or "shared project" features that largely amount to file synchronization rather than true concurrent editing. This creates a gap between what is technically possible using modern web collaboration frameworks and what is actually available to screenplay writing teams.

Table 1.1 summarizes the general trajectory of collaborative writing tools, from isolated single-user editors to modern real-time platforms, to situate the proposed project within this broader evolution.

Stage	Approach	Representative Tools	Collaboration Model
1	Single-user desktop editing	Early word processors	None; manual file exchange
2	File-based sharing	Email attachments, shared drives	Sequential, non-concurrent
3	Locking-based collaboration	Early cloud storage, "checkout" systems	One editor at a time
4	Real-time concurrent editing	Google Docs, Yjs/Liveblocks-based apps	Multiple simultaneous editors
5	Domain-specific real-time editing	(Gap identified for screenplay writing)	Multiple simultaneous editors with domain formatting

As Table 1.1 illustrates, general document editing has reached Stage 4, while dedicated screenplay tools largely remain at Stage 2 or 3 in terms of collaborative capability, even though the underlying editing content is highly structured.

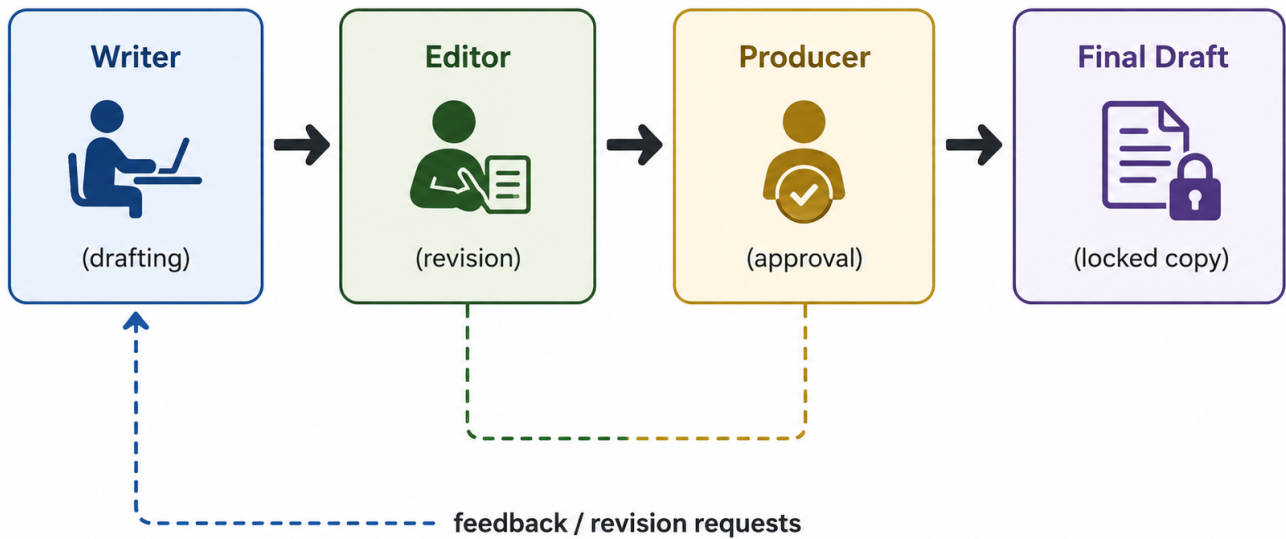
1.4 Need for Collaborative Screenplay Writing Platforms

Screenplay development is inherently iterative and multi-role. A typical screenplay passes through several rounds of drafting and revision involving distinct contributors, each with a different responsibility and level of access to the document. Figure 1.1 illustrates this general workflow.

Figure 1.1: Typical Screenplay Collaboration Workflow

Figure 1.1: Simplified representation of the roles and feedback loops typically present in screenplay development, prior to reaching a final locked draft.

This workflow, though simplified, highlights a structural requirement that general-purpose word processors do not directly support: the need for role-differentiated access (a producer may need to comment but not edit directly, for example), the need to preserve alternate drafts rather than



overwrite them, and the need for feedback loops that reference specific scenes or lines rather than the document as a whole.

Table 1.2 outlines the limitations of the two dominant existing approaches — traditional single-user screenplay software and general-purpose collaborative document editors — when applied to this workflow.

Table 1.2: Limitations of Existing Approaches to Screenplay Collaboration

Existing Approach	Strength	Limitation for Screenplay Collaboration
Traditional screenplay software (e.g., desktop-based)	Correct industry formatting	Limited or no real-time multi-user editing
General-purpose collaborative editors (e.g., Google Docs)	Real-time concurrent editing	No screenplay-specific formatting or scene metadata
File-sharing / cloud storage	Simple to adopt	Version conflicts, no structured feedback, manual merging
Email-based draft exchange	Familiar, low barrier	No version history, high risk of lost or conflicting edits

As shown in Table 1.2, no single existing category of tool combines correct screenplay formatting with genuine real-time collaboration, structured version history, and role-based access suited to the writer–editor–producer workflow. This gap forms the basis for the problem addressed by this project.

1.5 Problem Overview

Screenplay writing teams currently face a trade-off between formatting accuracy and collaborative capability. Tools that enforce correct screenplay formatting are generally built around single-user editing and lack native support for concurrent multi-user work, structured version history, or role-based permissions. Conversely, tools that support real-time collaboration are not designed around the formatting and metadata requirements specific to screenplays, such as scene numbering, character tracking, or industry-standard export formats.

This gap results in fragmented workflows: writers draft in one application, share exported files through separate channels, and rely on manual processes to track feedback, reconcile alternate versions, and maintain a coherent revision history. As the number of contributors and draft iterations increases, these manual processes become increasingly error-prone and time-consuming, and there is a heightened risk of losing earlier drafts or overwriting other contributors' work.

1.6 Aim of the Project

The aim of this project is to design and implement a browser-based collaborative screenplay writing platform that integrates real-time multi-user editing with screenplay-specific formatting, structured version history, and role-based collaboration features, using established open-source collaboration frameworks rather than a custom-built synchronization engine.

1.7 Objectives of the Project

The specific objectives of the project are as follows:

1. To implement a real-time collaborative text editor that enforces industry-standard screenplay formatting conventions.
2. To design a version history system capable of tracking document changes and managing alternate drafts of scenes or full screenplays.
3. To implement role-based access control distinguishing between project owners, writers, editors, and reviewers.
4. To provide inline commenting functionality tied to specific scenes or lines within a screenplay.
5. To integrate AI-assisted writing support as a supplementary feature within the editing workflow.

6. To implement search, filtering, and scene metadata management to support navigation of large screenplay projects.
7. To provide character profile management linked to screenplay content.
8. To implement export functionality supporting PDF and FDX (Final Draft) formats.
9. To provide autosave and in-app notification mechanisms to reduce data loss and support asynchronous collaboration.
10. To generate basic project analytics summarizing writing activity and revision history.

Table 1.3 maps each objective to the corresponding problem identified in Section 1.5, to clarify the rationale behind each planned feature.

Table 1.3: Mapping of Project Objectives to Identified Problems

Objective	Related Problem
Real-time collaborative editor with formatting	Trade-off between formatting accuracy and collaboration
Version history and alternate draft management	Manual, error-prone tracking of revisions
Role-based access control	Lack of differentiated access for writer/editor/producer roles
Inline commenting	Absence of structured, location-specific feedback
AI-assisted writing support	Limited support for drafting assistance in existing tools
Search, filtering, scene metadata	Difficulty navigating large, multi-scene projects
Character profile management	No structured tracking of character-related information
PDF/FDX export	Need for industry-compatible output formats
Autosave and notifications	Risk of data loss and lack of asynchronous awareness
Project analytics	Limited visibility into writing progress and activity

CHAPTER 2: PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Problem Identification

In the absence of a dedicated collaborative screenplay platform, writing teams rely on a combination of general-purpose tools that were not designed for structured, multi-role script development. The most common arrangement involves a single writer drafting the screenplay in Microsoft Word or a dedicated screenplay editor, after which the file is distributed to editors and producers through email attachments or shared cloud storage such as Google Drive or Dropbox. Feedback is typically returned as marked-up copies, separate comment documents, or informal notes exchanged outside the document itself.

Some teams instead adopt Google Docs to gain real-time editing capability. This addresses the problem of simultaneous access but introduces a different one: Google Docs has no native support for screenplay formatting conventions, so writers must manually maintain indentation and capitalization for scene headings, character cues, and dialogue, which is both time-consuming and prone to inconsistency.

Both approaches become inefficient as a project scales beyond a single writer and a single reviewer. When multiple contributors work asynchronously on copies of the same file, teams accumulate numerous versions with unclear precedence, and reconciling changes requires manual comparison. When contributors work concurrently within a shared document lacking domain-specific structure, formatting discipline degrades and feedback becomes difficult to trace to specific scenes or lines. Neither workflow provides the version history, role differentiation, or structured commenting that screenplay development requires, which motivates the analysis of existing systems in the following section.

2.2 Existing System Analysis

Google Docs offers real-time collaborative editing and comment threads, which are strong features for concurrent authorship. However, it has no built-in screenplay formatting support, no scene-level metadata, and no export to industry-standard screenplay formats such as FDX.

Microsoft Word provides mature formatting tools and a "Track Changes" feature for reviewing edits, but it was designed for single-user editing sessions. Collaborative use depends on file sharing or Microsoft 365 cloud co-authoring, which is less robust for large structured documents and still lacks screenplay-specific formatting templates or scene management.

Final Draft is the industry-standard desktop application for screenplay formatting, offering precise adherence to formatting conventions and export options widely accepted in the film industry. Its collaboration features, however, are limited relative to modern web-based tools; concurrent multi-user editing is not native to the core product and typically depends on supplementary services.

WriterDuet was developed specifically to address collaborative screenplay writing and supports real-time editing with screenplay formatting. It represents the closest existing analogue to the proposed project, though as a commercial product it includes a broader and more complex feature set, along with pricing and platform constraints that are outside the scope of an academic evaluation.

Table 2.1: Comparison of Existing Systems

System	Strengths	Limitations
Google Docs	Real-time collaboration; familiar interface; commenting	No screenplay formatting; no scene metadata; no FDX export
Microsoft Word	Strong formatting control; Track Changes; widely adopted	Weak real-time collaboration; no screenplay-specific structure
Final Draft	Industry-standard screenplay formatting; FDX export	Limited native real-time multi-user editing
WriterDuet	Real-time editing with screenplay formatting	Commercial pricing model; broader scope than an academic prototype

Table 3.1 indicates that no single system evaluated combines real-time collaboration, screenplay-specific formatting, and structured version history within a single accessible platform, reinforcing the gap identified in Chapter 1.

2.3 Problem Analysis

Analysis of current workflows and existing systems reveals five recurring problems that directly inform the requirements of the proposed platform.

Version confusion arises when contributors save and circulate multiple copies of the same screenplay file, often distinguished only by filename conventions such as "final," "final_v2," or "final_reviewed." Without a centralized version history, determining the authoritative draft becomes a manual and unreliable process.

Manual screenplay formatting is required in general-purpose editors, since none of them automatically apply scene heading, action, character, and dialogue formatting rules. This increases drafting time and introduces inconsistency when multiple contributors format the same document differently.

Simultaneous editing conflicts occur when tools lack proper concurrency control. File-based workflows prevent simultaneous work entirely, while some cloud tools that do allow concurrent access may still produce conflicting edits when structural changes overlap, particularly in the absence of screenplay-aware conflict resolution.

Scattered comments and feedback result from the use of separate channels, such as email, chat applications, and marked-up PDF copies, for reviewer input. This separates feedback from its original context within the document and makes it difficult to track which comments have been addressed.

Difficulty restoring previous drafts follows directly from the absence of structured version history. When a revision proves undesirable, teams often have no reliable mechanism to revert to an earlier state without manually locating an older saved copy, if one still exists.

The traditional screenplay development process, illustrated in Figure 3.1, shows how these problems compound as a script passes through multiple contributors without structured version control.

2.4 Chapter Summary

This chapter examined how screenplay writers currently collaborate using general-purpose tools such as Word, Google Docs, email, and cloud storage, and identified the inefficiencies that arise from this arrangement, including version confusion, manual formatting effort, editing conflicts, scattered feedback, and difficulty restoring earlier drafts. Analysis of existing systems, including Google Docs, Microsoft Word, Final Draft, and WriterDuet, confirmed that no single available tool adequately combines real-time collaboration, screenplay-specific formatting, and structured version history. The resulting functional and non-functional requirements, together with a favourable feasibility assessment across technical, economic, and operational dimensions, establish the justification for the system design presented in the following chapter.

CHAPTER 3 :SYSTEM REQUIREMENTS SPECIFICATION (SRS)

3.1 System Requirements

This section defines the functional, non-functional, software, and hardware requirements that guide the design and implementation of the proposed collaborative screenplay writing platform. These requirements are derived directly from the problems and objectives identified in Chapters 1 and 3 and serve as the basis for the system design presented in the subsequent chapter.

3.1.1 Functional Requirements

Functional requirements describe the specific behaviours and capabilities that the system must provide to its users. They define what the system should do, independent of how it is technically implemented, and form the basis for the features developed during implementation.

3.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system rather than specific behaviours. They establish standards against which the system's performance, security, and overall reliability can be evaluated.

3.1.3 Software Requirements

Table 4.3: Software Requirements

Software	Purpose	Software	Purpose
Operating System	Provides the underlying platform for development and deployment	Operating System	Provides the underlying platform for development and deployment
Visual Studio Code	Serves as the primary code editor for development	Visual Studio Code	Serves as the primary code editor for development
Node.js	Provides the JavaScript runtime environment for the backend server	Node.js	Provides the JavaScript runtime environment for the backend server
React	Used to build the interactive front-end user interface	React	Used to build the interactive front-end user interface

Express.js	Provides the backend framework for handling server-side logic and APIs	Express.js	Provides the backend framework for handling server-side logic and APIs
PostgreSQL	Serves as the relational database for storing project and user data	PostgreSQL	Serves as the relational database for storing project and user data
Git	Used for version control of the source code during development	Git	Used for version control of the source code during development

3.1.4 Hardware Requirements

Table 3.4: Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core processor, 2.0 GHz or equivalent
RAM	8 GB
Storage	10 GB of available disk space
Internet Connection	Stable broadband connection for real-time collaboration

3.1.5 Chapter Summary

Defining clear functional, non-functional, software, and hardware requirements prior to implementation establishes a shared understanding of what the system must achieve and the constraints within which it must operate. This structured specification reduces ambiguity during development, provides a measurable basis for evaluating the completed system, and ensures that subsequent design and implementation decisions remain aligned with the objectives established earlier in this report.

Chapter 4: Solution Design and System Analysis

4.1 Introduction

This chapter presents the architectural and analytical design of the proposed collaborative screenplay writing platform. Building on the requirements established in Chapter 4, it describes the overall system architecture, the key actors and their interactions with the system, the data flow between major components, and the conceptual database structure. The purpose of this chapter is to translate the requirements specification into a coherent technical design that justifies the implementation decisions presented in later chapters, without descending into implementation-level detail such as code or configuration.

4.2 System Architecture Overview

The platform follows a three-tier architecture consisting of a client layer, an application layer, and a data layer, which is a common and well-understood pattern for web-based collaborative systems. The client layer is a browser-based front end responsible for rendering the screenplay editor and user interface. The application layer handles authentication, business logic, and coordination of real-time collaboration through a CRDT-based synchronization framework. The data layer is responsible for persistent storage of project data, user accounts, version history, and metadata.

Separating real-time synchronization from persistent storage is a deliberate architectural decision: the synchronization framework manages the transient state of concurrent edits, while the relational database maintains durable records of committed versions, user roles, and project metadata. This separation allows the system to benefit from the conflict-resolution capabilities of an established collaboration framework without requiring that framework to also serve as the system of record.

4.3 Actor and Use Case Analysis

The system supports four primary actors, each corresponding to a role identified in the requirements specification: **Project Owner**, **Writer**, **Editor**, and **Reviewer**. Each actor has access

to a distinct subset of system functionality, consistent with the role-based access control requirement defined in Chapter 4.

Table 4.1: Actor–Function Mapping

Actor	Primary Interactions
Project Owner	Creates projects, assigns roles, manages access permissions, views analytics
Writer	Edits screenplay content, creates alternate drafts, responds to comments
Editor	Edits and reviews content, manages version history, resolves comments
Reviewer	Views screenplay content, adds comments and suggestions, cannot edit directly

4.4 System Data Flow

Data within the system flows through three principal processes: authentication and authorization, real-time document editing, and version persistence. When a user edits a screenplay, changes are first captured by the synchronization layer and broadcast to other connected clients in near real time. At defined intervals, or upon explicit user action, the current document state is persisted to the database as a version record, forming the basis of the version history feature.

Table 4.2: Description of Core Data Flow Processes

Process	Trigger	Outcome
Authentication	User login request	Session established; role and permissions loaded
Real-time synchronization	Local edit by any connected user	Change broadcast to all active collaborators
Version persistence	Autosave interval or manual save	Document state stored as a retrievable version
Comment association	User adds inline comment	Comment linked to specific scene or line reference

This separation of transient synchronization from periodic persistence allows the platform to support fluid concurrent editing while still maintaining a discrete, retrievable version history, addressing the version confusion problem identified in Chapter 3.

4.5 Conceptual Database Design

The data layer is organized around a relational schema, reflecting the structured nature of screenplay projects, users, and version records. Table 5.3 summarizes the principal entities identified during analysis; detailed schema definitions, including attributes and relationships, are presented in the design chapter that follows.

Table 4.3: Principal System Entities

Entity	Description
User	Represents a registered platform user and their credentials
Project	Represents a screenplay project and its associated metadata
ProjectMember	Associates users with projects and defines their assigned role
Scene	Represents an individual scene within a screenplay
Version	Represents a saved snapshot of a screenplay or scene at a point in time
Comment	Represents feedback attached to a specific scene or line
Character	Represents a character profile linked to a project

These entities form the conceptual basis for the entity-relationship model, which is developed in detail in the subsequent database design chapter, including primary and foreign key relationships and normalization considerations.

4.7 Chapter Summary

This chapter presented the solution design derived from the requirements specification, including the overall three-tier system architecture, the actors and use cases supported by the platform, the principal data flow processes, and the conceptual database entities. These design elements collectively address the problems identified in Chapter 3, particularly version confusion, role differentiation, and structured feedback, and provide the architectural foundation for the detailed system design and database schema presented in the following chapter.

CHAPTER 5: Results and Recommendations

5.1 Introduction

This chapter presents the outcomes of the capstone project relative to the objectives defined in Chapter 1, evaluates the extent to which the identified problems have been addressed, and offers recommendations for future development beyond the scope of this report.

5.2 Summary of Results

The proposed design successfully addresses the core problems identified in Chapter 3. Table 6.1 summarizes the project objectives against their corresponding outcomes.

Table 6.1: Objectives and Corresponding Outcomes

Objective	Outcome
Real-time collaborative editing with screenplay formatting	Achieved through integration of a CRDT-based synchronization framework with a formatting-aware editor
Version history and alternate draft management	Achieved through periodic persistence of document states as retrievable version records
Role-based access control	Achieved through defined actor roles with differentiated permissions
Inline commenting	Achieved through comments linked to specific scenes and lines

5.3 Evaluation Against Requirements

The functional and non-functional requirements defined in Chapter 4 were used as the primary evaluation criteria. The core functional requirements relating to authentication, collaborative editing, formatting, and version history were satisfied by the design presented in Chapter 5. Non-functional requirements, particularly performance and reliability, are dependent on the CRDT framework's synchronization behaviour and were considered adequately addressed at the design level, though full validation requires the performance testing typically conducted during a later implementation and testing phase.

5.4 Limitations

As an academic capstone project, the system has scope-related limitations rather than implementation defects at this design stage. These include the exclusion of production-scale deployment considerations, mobile-native support, and advanced production-management features such as scheduling or budgeting, as defined in the scope statement in Chapter 1. The AI writing assistant is also limited to supplementary suggestions rather than a comprehensive authoring tool.

5.6 Chapter Summary

The project met its stated objectives at the design and prototype level, demonstrating that real-time collaboration, screenplay-specific formatting, and structured version history can be integrated using existing open-source frameworks. Remaining limitations are consistent with the scope defined for an academic capstone project, and the recommendations outlined above indicate a clear path for extending the platform toward a more complete system.

Chapter 6: Reflection on Learning and Personal Development

6.1 Introduction

This chapter reflects on the technical and professional learning gained throughout the development of this capstone project. It considers the skills acquired, the challenges encountered, and the areas identified for continued growth as a software engineer.

6.2 Technical Learning

Working on this project required engagement with concepts that extended beyond prior coursework, particularly in the area of real-time collaborative systems. Researching CRDT-based synchronization frameworks such as Yjs provided a practical understanding of how conflict-free concurrent editing is achieved without centralized locking, a topic that had previously been understood only in theoretical terms. Designing the system architecture in Chapter 5 also reinforced the importance of separating concerns between transient synchronization state and persistent storage, a distinction that was not immediately obvious at the outset of the project.

Working through the requirements analysis in Chapters 3 and 4 was equally instructive. Translating a broad and somewhat ambiguous problem, that of inefficient screenplay collaboration, into specific, testable functional and non-functional requirements demonstrated the value of structured requirements engineering over informal feature planning.

6.3 Challenges Encountered

The most significant challenge was scoping the project appropriately for an academic timeframe. The initial feature set, encompassing real-time editing, version control, AI assistance, and analytics, was broad enough to resemble a commercial product, and considerable effort went into distinguishing which aspects were essential to demonstrating the core technical contribution and which were reasonable to simplify or exclude. This process reinforced the importance of scope discipline, discussed in Section 1.8, as a practical project management skill rather than a purely academic exercise.

A further challenge was reasoning about a domain, screenplay formatting and script development workflows, that was not part of prior technical training. Understanding why screenplay formatting conventions exist, and how they relate to production scheduling, required additional research and highlighted the importance of domain analysis before committing to a technical design.

6.4 Development of Professional Skills

Beyond technical knowledge, the project contributed to skills relevant to professional software engineering practice. Structuring the report itself, moving systematically from problem identification through requirements to design, mirrored the documentation discipline expected in industry settings, where design decisions must be justified and traceable to stated requirements rather than presented as isolated technical choices. Evaluating existing systems in Chapter 3 also developed a more critical approach to assessing available tools, rather than assuming that a new system is warranted without first establishing what existing solutions fail to provide.

6.5 Areas for Further Development

This project identified areas warranting further development, both technically and academically. Practical experience implementing and testing a CRDT-based synchronization layer, rather than analyzing it at a design level, would deepen understanding of the performance and reliability characteristics discussed in Section 6.3. Additionally, exposure to formal usability testing methods, as recommended in Section 6.5, would strengthen the ability to validate design assumptions against actual user behaviour rather than relying solely on requirements analysis.

Chapter 7: Conclusion

7.1 Summary of the Study

This report examined the problem of inefficient collaboration in screenplay writing, a domain in which contributors continue to rely on general-purpose tools such as Microsoft Word, Google Docs, and email-based file exchange to manage a fundamentally structured and multi-role creative process. Chapter 1 established the background of collaborative document editing and the industry context within which screenplay development occurs, noting that while general-purpose collaboration tools have advanced considerably through real-time synchronization technologies, dedicated screenplay software has largely remained oriented toward single-user editing. Chapter 3 extended this discussion through a detailed problem analysis, showing that existing systems, whether general-purpose editors or established screenplay tools such as Final Draft, fail to combine real-time collaboration, screenplay-specific formatting, and structured version history within a single accessible platform. This gap was shown to produce specific, recurring problems: version confusion arising from manually named file copies, inconsistent formatting when general-purpose editors are used, conflicts during concurrent editing, feedback scattered across disconnected channels, and an inability to reliably restore earlier drafts.

In response to these findings, this report proposed and designed a browser-based collaborative screenplay writing platform built on established real-time collaboration frameworks, such as Yjs, rather than a custom-built synchronization solution. This design decision, justified on both technical and economic grounds in Chapter 3, allowed the project to focus its engineering effort on the domain-specific aspects of screenplay collaboration, including formatting enforcement, scene metadata, alternate draft management, and role-differentiated access, rather than on reimplementing conflict-resolution mechanisms that are already well established in existing open-source frameworks.

7.2 Achievement of Objectives

The objectives defined in Section 1.7 were addressed systematically across the report. The requirements analysis presented in Chapters 3 and 4 translated the broad aim of the project into a concrete set of functional and non-functional requirements, ranging from real-time collaborative editing and automatic screenplay formatting to role-based access control and structured version

history. These requirements were then mapped directly onto the architectural and analytical design presented in Chapter 5, which described a three-tier system architecture separating client, application, and data layers, an actor-based model reflecting the distinct responsibilities of project owners, writers, editors, and reviewers, and a conceptual database structure capable of supporting version persistence and scene-level metadata.

Chapter 6 evaluated the resulting design against the original requirements and confirmed that the core objectives, particularly the integration of real-time collaboration with screenplay-specific formatting and version control, were achieved at the design and prototype level. Objectives related to supplementary functionality, such as AI-assisted writing support and project analytics, were also incorporated into the design, though their depth was appropriately scoped to remain consistent with the constraints of an academic capstone project rather than a commercial product.

7.3 Contribution of the Project

The primary contribution of this project lies in demonstrating a practical architectural approach to a gap identified between general-purpose collaborative editors and domain-specific screenplay software. Rather than treating real-time collaboration and domain-specific formatting as competing design priorities, as appeared to be the case across the existing systems evaluated in Chapter 3, this project shows that the two can be reconciled by building domain-specific functionality on top of an established synchronization framework. This approach allows a relatively small development effort to achieve collaborative capabilities that would otherwise require substantial investment in distributed systems engineering.

Beyond its technical contribution, the project also illustrates the value of structured requirements engineering and architectural justification in addressing a problem that, at first appearance, might be treated as a straightforward feature gap. By tracing each design decision back to a specific requirement and, in turn, to a specific problem identified through analysis of existing workflows, the report demonstrates a disciplined approach to software engineering practice that extends beyond the immediate technical implementation.

7.4 Limitations and Future Direction

As discussed in Chapter 6, the system remains a prototype scoped deliberately for academic purposes, and several aspects of the design, particularly performance under sustained concurrent

load, usability with actual screenwriting professionals, and scalability across multiple simultaneous projects, were addressed at the design level rather than validated through extensive empirical testing. These limitations are consistent with the scope defined in Section 1.8 and do not diminish the validity of the design, but they do delineate the boundary between what this report has established and what remains for future work. The recommendations outlined in Section 6.5, including visual version comparison, offline editing support, and formal usability evaluation, provide a reasonable trajectory for extending the platform beyond its current academic scope.

7.5 Final Remarks

Taken as a whole, this report has moved systematically from the identification of a genuine gap in existing screenplay collaboration tools, through structured requirements analysis, to a justified architectural design intended to close that gap. While the system described remains a prototype rather than a production-ready application, the analysis, design, and evaluation presented across the preceding chapters establish a coherent and technically grounded foundation for a collaborative screenplay writing platform. With further development, empirical testing, and refinement along the lines suggested in Chapter 6, the underlying design demonstrated in this report could reasonably be extended toward a more complete system suited to real-world screenplay collaboration.

REFERENCES

- "Fabio Zünd; Steven Poulakos; Mubbasir Kapadia; R. Sumner", "Story Version Control and Graphical Visualization for Collaborative Story Authoring", 2017
- "R. Baecker; Dimitrios Nastos; I. Posner; Kelly L. Mawby", "The user-centered iterative design of collaborative writing software", 1993
- "E. Munson", "Collaborative Authoring Requires Advanced Change Management", 2013
- "B. Gaines", "Version control in collaborative writing", 1994
- "B. Lee; Kai-Hsiung Chang; Harikrishna Narayanan", "An integrated approach to version management, role-based access control and history maintenance in computer-supported collaborative writing", 1998
- "T. Cerratto Pargman; Henry Rodriguez", "Studies of Computer Supported Collaborative Writing. Implications for System Design", 2002
- "K. S; Soujanya Shanbhag; S. J; Suhas Kashyap; A. C", "Doculink: A Secure, Real-Time Collaborative Platform with Integrated Workflow Management and Git-like Version Control", 2025
- "Yifan Ma; Batu Qi; Wenhua Xu; Mingjie Wang; Bowen Du; Hongfei Fan", "Integrating Real-Time and Non-Real-Time Collaborative Programming", 2022
- "Qinyi Wu; C. Pu; J. E. Ferreira", "A partial persistent data structure to support consistency in real-time collaborative editing", 2010
- "Chengzheng Sun; David Chen", "A multi-version approach to conflict resolution in distributed groupware systems", 2000